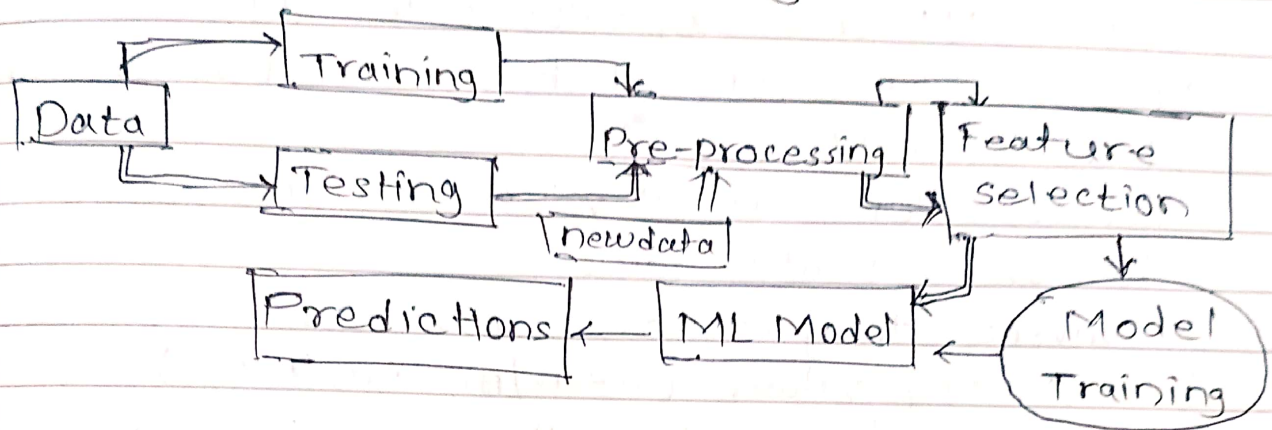


Machine Learning



Training → Find patterns & mapping

Testing → Unknown or user inputs are used

Two types of Training & Testing

1) Train test split

Training - 60 % (random)

Testing - 40 % (random)

Disadv. - Performance depends on patterns of data in Training & Testing

Stratified Training & Testing

Balances the target class in training & test data

Eg. We have 10 records

$Y == 0$

Training - 60% - random 3 records

Testing - 40% - random 2

Same for $Y == 1$

Disadv. : Model performance depends on patterns of data in training and testing.

2) K-fold cross validation

Split in k parts

Training → k-1

Testing → 1 randomly selected part

Final result = average of all k iterations of testing

Adv. - 1) More accurate

2) Reduce overfitting

Disadv - 1) Computationally expensive

2) Time consuming

Preprocessing steps

- 1) Replace 'NA' or missing value with "np.nan"
- 2) Remove columns/rows which have missing values more than threshold.
- 3) Remove column if all values are same.
- 4) Remove repeated rows
- 5) Convert string columns $\rightarrow$ numbers
- 6) Fill missing values (mean/median/mode)
- 7) Normalize/Standardize the numeric columns
- 8) Format/Transform the target (if required)

Normalization $\rightarrow$ Rescale data to specific range

Standardization $\rightarrow$ mean = 0 & std dev. (σ) = 1

Feature Selection

- Better features : improve performance
- Irrelevant features : increase complexity & computational cost

Methods



Unsupervised

- PCA
- Kernel - PCA
- ICA
- Boruta Algorithm



Supervised

- Model based
- Regularization based

Unsupervised Feature Selection

- 1) Principle Component Analysis (PCA)
 - Create components based on Eigen values and Eigen vectors.
 - Components are created using linear relationship in all columns.

- Few components can be selected based on variance addressed by every component.

- Advantage:

- Un-supervised way of feature reduction.

- Disadvantage:

- Addresses only linear relationship in data.

- Selecting few components may lead to information loss.

2) Kernel - PCA

Kernel PCA is a non-linear extension of traditional PCA in machine learning used for dimensionality reduction by mapping data into a higher-dimensional feature space via kernel functions, where linear PCA can then separate complex patterns effectively.

Kernel functions:

- RBF Kernel (Radial Basis function kernel)/Gaussian kernel

- Models similarity between different points:

$$\text{similarity} = e^{-\gamma \|x_1 - x_2\|^2}$$

$\|x_1 - x_2\| \rightarrow$ distance between any two points in the data

$\gamma \rightarrow$ radius of input

- addresses non-linear relationship.

- Polynomial Kernel

- We create polynomial features,

$$K(x_1, x_2) = (\gamma \cdot x_1 \cdot x_2 + c)^d$$

$d =$ degree of polynomial

$\gamma =$ coeff. of impact

- Cosine Kernel

- In cosine kernel we ~~apply~~ measure cosine of the angle

between two points.

$$K(x, y) = \frac{x \cdot y}{\|x\| \cdot \|y\|} = \cos \theta \quad (\theta \rightarrow \text{angle b/w them})$$

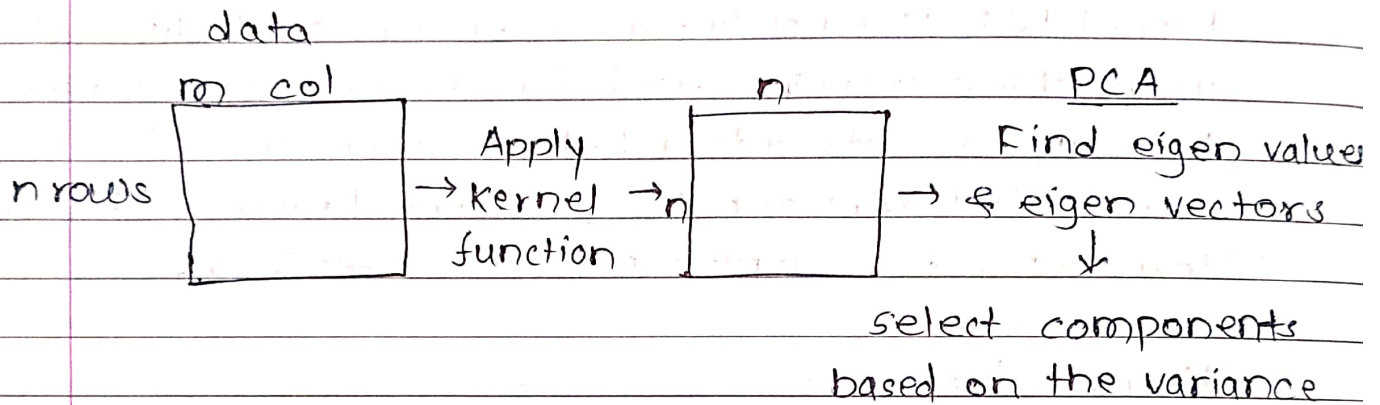
- Mostly used with language data.

• Linear Kernel

$$K(x, y) = x \cdot y$$

- Same as normal PCA

Steps in kernel PCA



Advantage:

Good for non-linear dimensionality reduction

Limitation:

In big data, computational cost is high.

Maximum component can be generated by PCA = Minimum no. of rows & columns.

How you select important components from PCA?

→ Decide total variance / information we need to keep.

eg. 95% info → select all components where total variance is 95% or more.

Supervised

1) Model based

► Select from Model

- Train a supervised model and extract feature importance.
- Select the top few features by threshold or by count
- Compatible Models (Classification / Regression)
 - Decision Tree, Random Forest, AdaBoost
 - Linear SVM
 - Logistic Regression (Only classifier)
 - LASSO, RIDGE (only Regression)
- Pass the threshold or pass max-features to select

► RFE: Recursive Feature Elimination

- 1) Step 1: Take a model
- 2) First evaluate model for all features
- 3) Eliminate a feature and check performance: remove feature if performance is constant or improves after removal.
- 4) Repeat step 3 for all features one by one.

- selected (i.e., estimated best) features are assigned rank 1.
- All other features are ranked differently.
- applicable to classification & regression models.

2) Regularization Based Feature Selection

• L1 Penalty (LASSO)

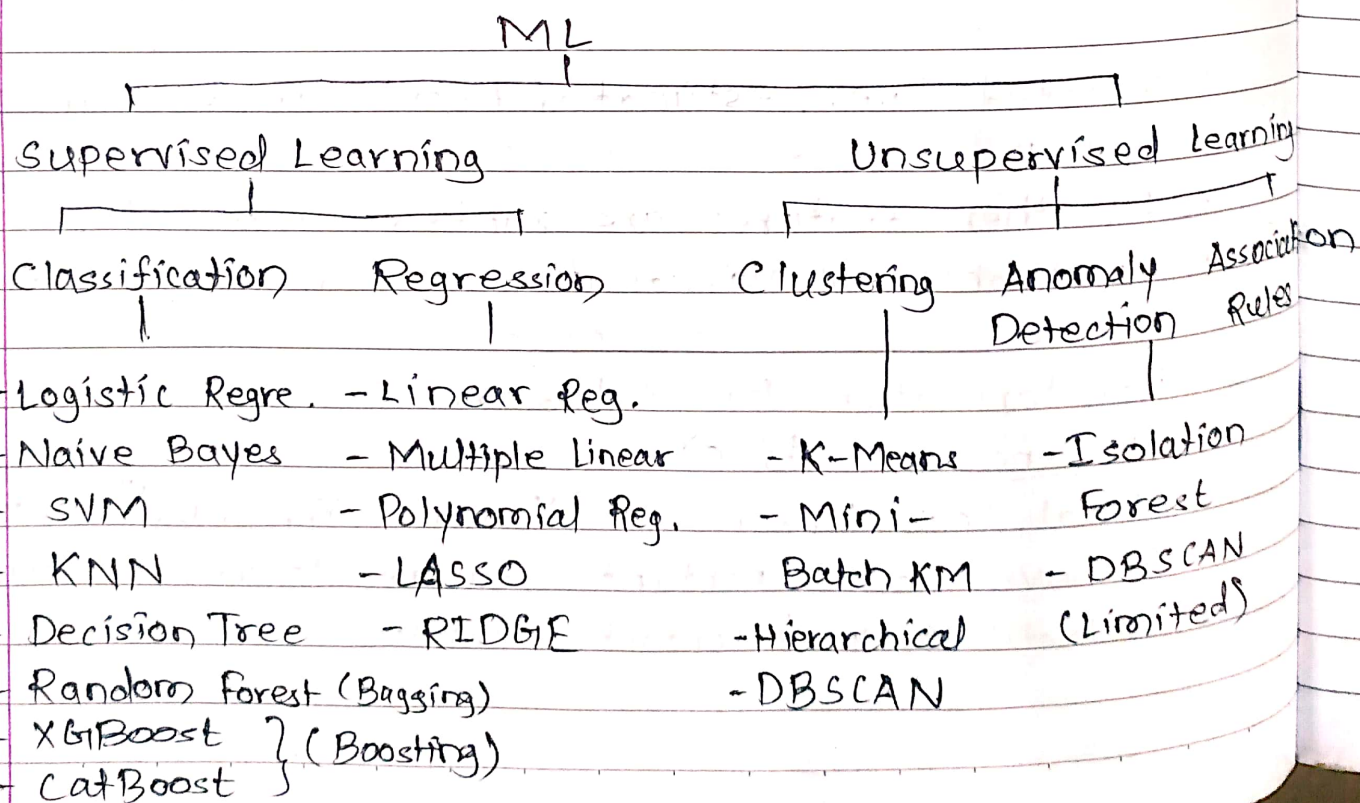
- Total Loss / Error = $\sum (Y_{\text{actual}} - Y_{\text{predicted}})^2 + \alpha \times \sum |\text{coeffs}|$
- Remove non-important features to reduce complexity
- Selects ONLY important features.

- L2 Penalty (Ridge)
- Total Loss/Error = $\sum (Y_{\text{actual}} - Y_{\text{pred}})^2 + \alpha \times \sum \text{coeff}^2$
- Make the coeff. value of features smaller to reduce complexity
- No feature is completely removed.
- ElasticNet - L1 & L2 Both

Model Training

- Model training is the process of teaching a machine learning model to make correct predictions by showing it examples from data.
- 1) Give the model input data
- 2) Provide correct answers (labels)
- 3) The model makes predictions
- 4) Calculate error (loss function)
- 5) Update model parameters (GD, Adam, RMSProp)
- 6) Repeat thousands of times (epoch)

ML Models



Classification Task

- Predict a class / label using features / columns
- Types of Classification
 - Binary Classification
 - Multi-Class Classification

Binary Classification

- Only Two classes in output
- Positive class (Represented by 1)
- When test results are positive
- Negative class (Represented by 0)
- When test results are negative.

Multi Class Classification

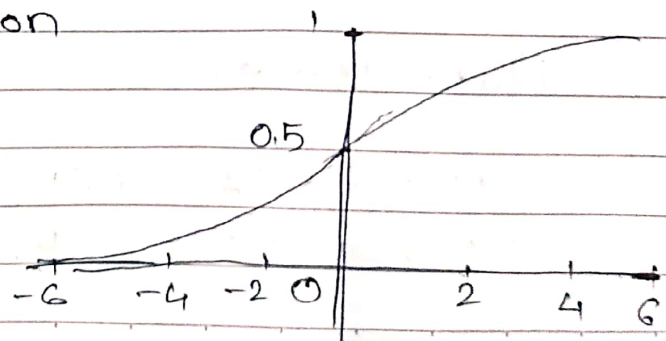
- More than two classes in output

Logistic Regression (logit classifier)

- Used for classification (binary & multi-class) only
- Models linear relation b/w independent variables to predict a binary or categorical o/p.
- Target is to have minimum loss, so it does find the right coeff. by doing multiple iterations.
- o/p → probability (b/w 0 to 1)
- $t = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n$
where b_0 = intercept, b_1 to b_n = coeffs.
- $Y = \text{sigmoid}(t) = 1/(1 + e^{-t})$
- converts the real valued t into value b/w 0 to 1.

Logistic / Sigmoid Function

- S-shaped curve
- I/p : $(-\infty, \infty)$
- o/p : $(0, 1)$
- Real → Binary



- Binary Classification
Output is directly converted to 0 or 1 using threshold (eg: < 0.5)
- Multi-Class Classification
- o/p is in form of probability of each n classes (softmax)
- o/p is normalized ($\sum$ probabilities of all classes = 1)
- Softmax Function : $P(Y=i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$ for i^{th} class from total K classes.

Logistic Regression Parameters

- Solver = 'saga', 'liblinear', 'lbfgs'
- According to Scikit Documentation : SAGA is best choice
- Penalty : l_1 , l_2 , elasticnet
- The 'liblinear' supports both L_1 & L_2 regularization
- Elastic-Net supported by 'saga' solver.
- 'saga' supports all penalties.
- class_weight : balanced or in dictionary form
- l1_ratio : used when penalty is 'elasticnet', tells how much portion will be l_1 penalty.
- multi_class : support for multi class classification problem
 - 'multinomial' : 'saga' solver supports this.
 - 'ovr' : One vs rest strategy supported by 'liblinear' solver.

Naive Bayes

$$P(AB) = P(A) \times P(B) \dots A \& B \text{ independent}$$

$$P(AB) = P(A) \times P(B|A) = P(B) \times P(A|B) \dots A \& B \text{ dependent}$$

Bayesian Model

$$P(C|x) = \frac{P(C) P(x|C)}{P(x)}$$

Naive Bayes

- Binary Classification Task ONLY
- I/P - independent
- Bayesian Probabilistic Model is used
- Simple, effective & scalable
- Application: Automatic medical cases classification

Types of Naive Bayes

- Gaussian Naive Bayes
 - numeric & large data (used for)
- Multi Nominal Naive Bayes
 - categorical features like Text data
- Bernoulli's Naive Bayes
 - Binary Variables
- Application of Naive Bayes/ Bayesian Formula
- Incremental learning, Probabilistic Reasoning

Support Vector Machines (SVM)

- Used ONLY for Binary Classification or Regression
- can handle linear / Non-linear boundary
- Goal: Find decision boundary b/w two classes with maximum margin.
- Margin = $\min$ distance b/w boundary closest sample.

Why max margin? → avoids misclassification, even if there is some deviation in new data from original data.

- Binary Classification!
- Data points are separated by a decision boundary.

- A new point is classified based on which side of the boundary it lies.
- Regression :
 - A linear equation forms the decision boundary.
 - The o/p is passed through a logistic (sigmoid) function.
 - The resulting probability is used for prediction.

SVM Kernel Trick

- Transform the data by adding non-linear features.
- Enables linear boundary
- Efficient only once not for every iteration.
- RBF Kernel : Radial basis Function / Gaussian Kernel
 - considers similarity between points in the dataset
 - similarity is based on gamma & distance b/w points.

SVM parameters

- C : regularization parameter (ℓ_2 penalty)
 - Higher value less regularization / more misclassification
 - Always positive
- Kernel : 'linear', 'rbf'
- gamma : Kernel coefficient for 'rbf'
- Less value : more support vectors are selected (slow)
 - influence is far/wide $\rightarrow$ model is smooth, behaves closer to linear
- High value : less support vectors are selected (fast)
 - influence is close/local $\rightarrow$ model is complex, risk of overfitting.
- Needs fine tuning
- gamma controls the radius of influence of a training point.

K Nearest Neighbors (KNN)

- Supervised or Unsupervised (binary & multiclass)
- Finds the nearest data point and stores neighbour data.
- classification - majority class of K neighbours
- regression - mean of K neighbours

Decision Trees

- Supervised Learning
- Can be used for classification / Regression
 - Binary & Multi-class both supported
- Weak algorithm - Easy to over-fit
- Popular Algorithms { ID3
 CART (Classification & Regression Trees)

Basic Terminologies

- Root Node: entire population & this further gets divided into two partitions.
- Splitting: dividing a node into two sub-nodes.
- Decision Node: When a sub-node splits into further sub-nodes, then it is called decision node.
- Leaf / Terminal Node: Nodes do not split
- Pruning: removing sub-nodes of a decision node
- Branch / Sub-Tree: Sub-section of entire tree
- Parent Node: A node, which is divided into sub-nodes.
- Child Node: sub-nodes are the child of parent node.

CART Algorithm

- CART at every step divides currently available rows in two parts in way that cost will be minimized (greedy approach)

- This is called as recursive binary splitting.
- Cost Function
 - For Classification : Gini Index / Entropy
 - For Regression : sum of squared error
- Stopping Criteria
 - Ideal : all rows in the part should have same class OR only 1 row in the part
 - Actual : min. samples - leaf parameters allows CART to stop splitting if no. of rows are less than minimum value given

CART for Classification

- Leaf nodes will have the predicted class
- supports Binary & Multi-class classification

Criteria for Split (cost function)

- Measures how each feature/column is important
- Options
 - Gini Index → CART uses it by default
 - Entropy (Information Gain)

Gini Index

- Uses Square of probability

$$\text{GiniIndex} = 1 - \sum_j P_j^2$$

(j = jth class)

- Fast

- Range : 0 to 0.5

- More used

Both select almost similar features

Entropy

- Uses Log of probability

$$\text{Entropy(Parent)} = - \sum_{i=1}^C P_i \log_2 P_i$$

Slow

Range : 0 to 1

Less Used

CART for Regression

- Leaf nodes have mean of the target column (prediction value)
- Mean of target from all rows present in leaf node is taken for prediction

Criteria for Split

- Data is split to reduce the variance (error) in the Target variable
- best feature - split which results in minimum variance

- Squared error: to reduce mean squared error

$$MSE = \frac{1}{n} \sum (Y_{actual} - Y_{predicted})^2$$

- Absolute Error: to reduce MAE

$$MAE = \frac{1}{n} \sum |Y_{actual} - Y_{predicted}|$$

Parameters in Decision Tree (Sklearn)

- min-samples-leaf
- Minimum number of samples that must be present in a leaf node.
- Split only if min samples are present in the branch
- Used for controlling over-fitting
- Values:
 - int → min number required
 - float → min fraction of samples required
- Best value needs to be searched by fine tuning
- max-features
 - The no. of features to consider when looking for the best split.

- If not set then consider all features for best split
- Which value to use?
 - 'int' : to select given features
 - If too many features then use 'sqrt' / 'log2'
- Best value : Needs to be fine tuned

Splitter for Every Node

- Way to search feature / column and split value for that column
- Options :
 - Best split
 - Consider all features (max-features) for current node
 - Select best out of all possible splits
 - Random split
 - Select random split from all possible splits for current node

max_depth

- Allowed depth of the tree
- Controls over-fitting
- Helps in pruning the tree
- No fixed 'best' value - need to fine tune the value

ccp_alpha

- Cost complexity pruning parameter
- Used to avoid over-fitting
- Greater value of ccp-alpha $\rightarrow$ more pruning

class_weight

- Generally, all classes in data are assigned same weightage
- Values :

- none : all classes same
- Balanced : Weights taken from training data, (used for imbalanced data)
- dict : User given values

CART Advantage and Dis-advantage

Advantages :

- 1) Easy to understand
- 2) Data Scaling is not required
 - Robust to different variables having different range of values
- 3) Doesn't need to have Normally distributed features
- 4) ~~Doesn't need to have Normally distributed feat~~
- 4) Provides feature importance, so can be used for feature selection in classification or regression
- 5) Can create non-linear boundaries easily

Dis-advantage :

- 1) Easy to Over-fit
 - solution : Ensemble learning
- 2) Slow when high Number of features
 - splitting takes time when features are many
- 3) In Regression, CART can't predict values beyond range of values it is trained on
- 4) Trees can have high depth
 - because, CART does only Binary splits

Overfitting

- Performance on training data increases but test data performance decreases after a threshold
- Model cannot generalise for new data

Solution to Overfitting

- Fine Tune min-samples-leaf
- Fine Tune depth of tree
- Use Cross Validation than just train test split
- Use Ensemble Learning than just single tree

Evaluation Metrics for Supervised Learning Classification Task

- Confusion Matrix

		Actual values	
predicted values	0	True Negative	False Positive
	1	False Negative	True Positive

$$\text{Recall} = \frac{TP}{(TP + FN)} \quad (\text{Sensitivity / True Positive Rate})$$

$$\text{Precision} = \frac{TP}{(TP + FP)}$$

$$F1 \text{ score} = \frac{2 \times \text{precision} \times \text{Recall}}{(\text{Precision} + \text{Recall})}$$

$$\text{Accuracy} = \frac{(TP + TN)}{\text{Total Samples}}$$

Recall → How many of the actual positive cases the model correctly identified (focuses on False Negative)

When to use: When missing positive cases is costly (eg. disease detection, fraud)

Precision → How many of the predicted positive cases were actually positive (focuses on False Positives)

When to use: When minimizing false alarms is critical (eg. spam detection)

F1 score $\rightarrow$ The harmonic mean of Precision and Recall, offering a single score that balances both, penalizing extreme values.

When to use: Excellent for imbalanced datasets or when you need a balance between minimizing False Positives and False Negatives.

Accuracy $\rightarrow$ The overall correctness of the model; the proportion of all correct predictions out of all predictions.

When to use: Good for balanced datasets where all classes are equally important.

- Log Loss (Cross-Entropy)
- Tells you how confident the model is in assigning a class to an observation
- A lower log loss value means better predictions.
- Log is taken to give
- less penalty to less difference
- And more penalty to more difference
- All values of log loss are negative
- $\min = -\inf(\log(0))$
- $\max = 0 (\log(1))$

Why is Log loss important?

- Unlike accuracy, it considers prediction confidence.
- Widely used in:
 - Logistic Regression
 - Neural Networks (classification)
- Helps evaluate probabilistic classifiers.

$$\text{Log loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$$

- ROC (Receiver Operating Characteristic Curve)
 - Graph of values of true positive rate and false positive rate
- AUC (Area Under Curve for ROC curve)
 - Area under ROC curve
 - ROC and AUC used when dataset is balanced,
- Precision Recall Curve
 - Graph of various precision and recall values for different thresholds
- Average Precision score
 - Area under the curve of Precision Recall Curve is known as average precision score.
- Usage
 - Precision Recall Curve and Average Precision is used when Dataset is imbalanced.

Regression

Linear Regression

- A supervised learning algorithm used to predict a continuous dependent variable using one independent variable.
- Eqⁿ : $y = B_0 + B_1 x$
- Loss Function : MSE
- Use cases : 1) salary vs experience
2) House price vs area

Key Assumptions

- Linear relationship
- No multicollinearity
- Homoscedasticity
- Normally distributed errors

Advantages :

- Simple and easy to interpret
- Fast to train
- Works well for linear relationships

Disadvantages :

- Cannot model non-linear data
- Sensitive to outliers
- Assumes linearity

■ Multiple Linear Regression

- A regression technique that predicts a continuous output using multiple independent variables.

Eqn :: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$

Advantages :

- Considers multiple factors
- More accurate than simple linear regression
- Interpretable coefficients

Disadvantages :

- Multicollinearity problem
- Assumption heavy
- Overfitting if features are many

- Use cases : House price prediction

Polynomial Regression

A regression method that models non-linear relationships by transforming features into polynomial terms.

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots$$

Advantages

- Handles non-linear data
- More flexible than linear regression
- Easy to implement

Disadvantages

- Overfitting risk
- High degree increases complexity
- Poor extrapolation

Use cases : 1) Growth curve modeling

2) Trend analysis in time-based data

Solution to complexity or overfitting of Model:

1) Lasso Regression (L1 penalty)

Lasso regression is a regularized regression technique that adds an L1 penalty, forcing some coefficients to become exactly zero.

$$\text{Loss} = \text{MSE} + \alpha \sum |\beta_i|$$

Advantages

- 1) Performs automatic feature selection
 - Produces sparse models
 - Improves model interpretability

Disadvantages

- Can drop important features if highly correlated
- Less stable with multicollinearity
- Sensitive to choice of α

Use Cases

- High dimensional data
- Feature selection tasks
- Text and gene expression data

2) Ridge Regression (L2 Regularization)

Ridge regression is a regularized linear regression technique that adds an L2 penalty to reduce model complexity and prevent overfitting.

$$\text{Loss} = \text{MSE} + \alpha \sum B_i^2$$

Advantages

- Reduces overfitting
- Handles multicollinearity well
- Keeps all features (coefficients shrink but don't become zero)

Disadvantages

- Does not perform feature selection
- Coefficients become harder to interpret
- Choice of α is critical

Use Cases

- Data with many correlated features
- Financial and economic models
- When all features are important

ElasticNet Penalty

ElasticNet Penalty
Uses combination of $L1$ & $L2$.
Because of $L1$ penalty using

Disadvantage: Because of L_1 penalty using coordinate descent elasticNet also doesn't guarantee best performance.

Regularization in decision tree → pruning
Pre-pruning post-pruning
 crrp - alpha

- Pre-pruning
- max_depth
- min_samples_leaf

Bias Complexity Tradeoff

Bias and Complexity (variance) in Prediction models

- Bias in Model :
- Bias is Assumptions
- More bias : simple model
- complexity (variance) in Model
- Variance is how much model changes based on different training samples
- More Variance : Complex Model

Components of Total Loss

- Total Loss / Error of Model
- Baseline Loss / Error - Irreducible Error
- Error due to Bias
- Error due to variance
- Total loss is sum of all of the above

Under-fit : When training loss and test loss both are high

Solution $\rightarrow$ Increase iterations of training

- Increase data $\rightarrow$ rows (samples) or columns (features)

Over-fit: When training loss ^{is high} and test loss ^{is low} both are

- solution $\rightarrow$
- Regularization (L1, L2 or Elastic-net)
 - Ensemble Learning

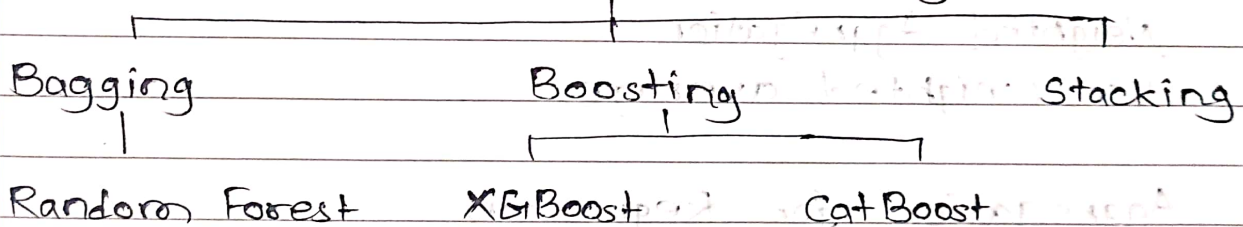
Bagging reduces variances.

Boosting reduces bias and variances.

Ensemble Learning

- Used for classification & Regression
- Combines multiple models together
- Stacking $\rightarrow$ o/p of one ML algo into another model.

Types of Ensemble Learning Models



i) Bagging (Bootstrap Aggregation)

i) Bootstrapping

- It is used to create multiple datasets from a single dataset using sampling.
- Here, fixed percentage of original data is taken & remaining samples are created using repetition.

ii) Model Training

- Use 'n' datasets created in bootstrapping to train 'n' models.

iii) Aggregation

Combine results of all models to give final result

Aggregation for classification

Every model does voting. Two types of voting

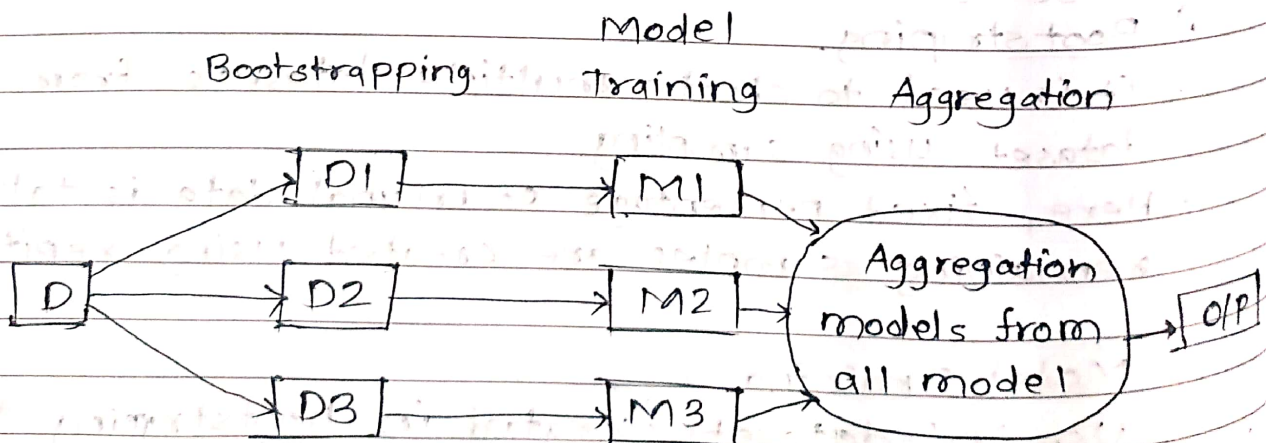
- 1) Soft voting - Every model gives probability of every class in output.
- 2) Hard voting - Every model gives class as output

Aggregation Types :

- Simple Aggregation :
 - Use 'mode' for hard voting : class with maximum votes is predicted
 - Use 'mean' for soft voting : class with highest 'mean' probability is predicted.
- Weighted Aggregation :
 - Use weighted average

Aggregation for Regression

We take avg. of output of all models (simple & weighted)



Random Forest Model

- One of the most popular model of bagging technique.
- Can be used for binary/multiclass classification or regression.
- All models are decision trees.

There are 2 improvements in random forest compare to bagging :

- 1) In bootstrapping, sampling is done ^{on} rows as well as columns.

Using max-features, we are going to sample columns.

- 2) Random forest uses 'soft voting' in aggregation step.

Advantages :

- 1) Two way randomization avoids the variance in model.
- 2) Robust model
- 3) Because it is based on bagging it can be trained using parallel computing.

Disadvantages :

- 1) More memory and time than linear models (slow for large data)
- 2) Not suited for text / sparse data.
- 3) In regression, predictions cannot go beyond range of training values.

Random Forest Parameters

- n-estimators : No. of decision trees to be created

- need to be fine tuned (use Gridsearch).
- max-samples :
 - How many samples to draw from original data in each new dataset.
 - generally set to 0.8 (80%) by default | (100%)
- max-features
 - No. of columns to be considered inside every decision tree split
 - Higher value, random forest classifier becomes more stable
 - If equal to all features then decision trees may overfit
 - If very low then deep decision trees are created not good.
- oob-score (Out of Bag Score) :
 - Each tree is evaluated using the data not used in its training (out-of-bag samples)
 - Acts as an internal validation metric or no need for separate validation set
 - True $\rightarrow$ model generalizes well on unseen data
 - False $\rightarrow$ model may be overfitting
- max-depth
 - maximum depth allowed for each decision trees.
 - smaller depth $\rightarrow$ simpler trees $\rightarrow$ less overfitting
 - larger depth $\rightarrow$ complex trees $\rightarrow$ more memory & overfitting
 - use : important when memory usage is a concern
- controls tree complexity and helps prevent overfitting

- n-jobs :
- Number of CPU cores used for training :
- 1 → Use all available cores → maximum parallel computing

Boosting Models

Boosting is an ensemble learning technique that builds a strong model by iteratively training multiple weak models and aggregating their predictions.

How Boosting Works

- Models are trained sequentially, not independently.
- In each iteration, the new model focuses on reducing the total error or loss made by previous models.
- Errors from earlier model influence how the next model is trained.

Bias-Variance Handling

- Repeated iterative training helps reduce bias, because each new model corrects mistakes of the previous one.
- Aggregation of multiple models helps reduce variance, improving generalization.

Learning Rate & Iterations

- Learning rate controls how much each new model contributes to the final prediction.
- High learning rate → larger changes per iteration
- Low learning rate → smaller, more controlled updates
- No. of iterations and learning rate must be carefully fine-tuned to avoid overfitting or underfitting.

Limitation

- Traditional Gradient Boosting trains models sequentially making it slow.
- Specialized libraries (XGBoost, LightGBM, CatBoost) are required for parallel & faster training.